

complexity of $O(\log N)$? Of course, it is the binary heap. You also know that you can build a binary heap in $O(N)$ time. Therefore, the solution is to build a binary heap from the array of N unsorted integers and perform ' k ' *extract_min* operations on the binary heap.

RAIL: Resource Acquisition Is Initialisation

This is a popular design pattern, used in C++ in order to ensure the release of resources acquired at the beginning of a function scope. Resources acquired in a particular scope should be released before leaving the scope, unless the ownership is being transferred to another scope or object explicitly so that the new owner will take the responsibility of releasing the resource after using it. Typically, this is achieved by having a resource acquire call at the beginning of the function (where the resource is going to be used) and a resource release call at the end of the function (after which you do not need the resource anymore). The resources can be memory, file, lock, etc. Very often you see a pair of function calls such as (*malloc*, *free*), (*mutex_lock*, *mutex_unlock*), (*fopen*, *fclose*) at the beginning and end of the function.

For a programmer, it is sometimes possible to forget writing the code that performs the resource release. If there are many paths of control from which the function can return, including some paths where you return due to errors, the programmer can forget releasing the resource along one of these paths. This can result in resource leakage.

Consider the following piece of code:

```
bool foo ()
{
    char * mystr = new char [100];
    if (cond1)
        do something //with acquired memory
    else if (cond2)
        return false;
    else
        throw "ERROR";

    // This may not be invoked in all return paths
    // hence memory can be leaked
    delete [] mystr;
    return true;
}
```

In the above snippet of code, there are control flow paths in the routine where the programmer has forgotten to free the memory acquired before returning from the function. This is a common programming error, often leading to memory leaks.

Consider the following piece of code:

```
bool bar ()
{
    pthread_mutex_lock(&mutex1);
    if (cond1)
```

```
do something
else if (cond2)
    return false;
else
    throw "ERROR";

// The lock release may not be invoked in all return paths
// hence the code end up in a deadlock
pthread_mutex_unlock(&mutex1);
return true;
}
```

In the above code snippet, there are paths of control flow where the programmer has forgotten to release the lock, which can lead to a possible deadlock. This is again a common programming error when the flow of control in the program is quite complex, as in the example above.

RAIL is a programming idiom/design pattern aimed at solving the problem of a missing resource release along all return paths from a given scope. The idea is to wrap the resource acquire in the constructor of an object whose scope is the same as the function scope. The destructor of the object contains the call to the resource release function. Since the destructor of the object will be called whenever it goes out of scope automatically, the resource release function will be called automatically whenever there is a control flow path returning from the function.

Note that it is not mandatory that the resources need to be acquired in the constructor of the object, though it is often convenient to do so. However, the key to RAIL is that the resources get released in the destructor of the object. Hence, RAIL is also sometimes known as Resource Release is Finalisation idiom. I leave it to the readers to come up with code that uses RAIL to avoid the resource leakage in the two examples of code snippets shown earlier. The resource release functions we have seen are 'free' and 'mutex_unlock' which do not throw exceptions. What would happen if the resource release function ends up throwing an exception? How would RAIL handle such cases?

I would like to request readers to send in their solutions to the remaining questions from last month's column. Also, if you are preparing for computer science programming interviews, you may find it useful to visit my programming interviews discussion group 'Computer Science Interview Training (India)' on LinkedIn.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me along with your solutions and feedback at sandysm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

About the author:

Sandya Mannarswamy, The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.